

ATIVIDADE_07_VICTOR_EMANUEL_SILVA_SANTANA_260143092

Expressões, Atribuição e Controle.

A importância da criação de um código limpo e legível, onde outras pessoas conseguirão interpretar de forma simples. Onde uma simples expressão, símbolo ou até mesmo a precedência que o código foi construído, pode fazer toda diferença para determinado resultado.

Módulo 1: Quiz de abertura

1. Python

palpite: 8

Resultado: -4 512

2. Java

Palpite: 12 3

Resultado: Main.java:1: error: class, interface, annotation type, enum, record, method or field expected

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

^

1 error

3. JavaScript

Palpite: 0

Resultado: 3 52

4. C

Palpite: 0

Resultado: <div><br class="Apple-interchange-newline">printf("%d\n", 3 > 2 > 1);
<br id="textarea-placeholder-break" aria-hidden="true"></div>

Exit 1

Main.c:1:8: error: expected declaration specifiers or '...' before string constant

```
1 | printf("%d\n", 3 > 2 > 1);
  |           ^~~~~~
```

Main.c:1:16: error: expected declaration specifiers or '...' before numeric constant

```
1 | printf("%d\n", 3 > 2 > 1);
```

5. Python

Palpite: true

Resultado: Padrão true

6. JavaScript

Palpite: 0

Resultado: true false true

Modulo 2: Expressões: precedência, efeitos colaterais, sobrecarga

Expressões aritméticas: questões de projeto

Quais são as regras de precedência?

R: Segue a regra do que será executado primeiramente, uma forma hierárquica de como o algoritmo será interpretado e sobre qual passo deve seguir primeiro. Neles estão inclusos: operadores como parenteses, aritméticos, relacionais, lógicos.

Quais são as regras de associatividade?

R: Gerencia e define regras matemáticas de como operadores podem ser interpretados quando não existe o complemento de um operador com outro.

Em que ordem os operandos são avaliados?

R: parenteses, operadores aritméticos, relacionais, lógicos.

Há restrições a efeitos colaterais?

R: Se a ordem de precedência não for seguida como regra, haverá efeitos colaterais no resultado esperado.

O usuário pode sobrecarregar operadores?

R: Depende estritamente da linguagem utilizada e qual o contexto para essa sobrecarga. Algumas linguagens como C++ permitem esse tipo de sobrecarga para testes ou para resultados específicos. Outras linguagens, como o próprio JAVA, procura sempre manter um código sóbrio e como uma fácil leitura, não dando essa liberdade para abusar da sobrecarga dos operadores.

Que mistura de tipos é permitida?

A conversão de operadores seria um exemplo dessas, em um caso esporádico onde é preciso converter um **int** para **string**, em algum contexto que for necessário apresentar tal número como string no final.

Precedência e associatividade na prática:

Resumo básico sobre ambas, precedência define qual operador deve ser utilizado e quando ser utilizado, associatividade é a ordem que esse operador será utilizado, qual operador atuará antes, qual operador atuará em seguida e qual a ordem de leitura que esse operador será interpretado, por exemplo: começando da esquerda ou começando da direita.

Ordem dos operandos e efeitos colaterais:

Pode variar de acordo com a linguagem utilizada, a lógica de cada linguagem define qual é sua ordem de precedência e se será utilizada essa regra ou não.

Operadores sobrecarregados:

Java:

```
System.out.println('a' + 1);           // 98
System.out.println("'" + 'a' + 1);    // a1
x.add(y).multiply(tres); // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, outro): # v1 + v2
        return Vetor(self.x + outro.x,
                       self.y + outro.y)
    def __mul__(self, k):      # v * 3
        return Vetor(self.x * k, self.y * k)
v = Vetor(1, 2) + Vetor(3, 4) # Vetor(4, 6)
print(v * 3)                  # Vetor(12, 18)
```

Seguindo os dois exemplos apresentados, é possível notar que o exemplo em java não dá liberdade para a utilização de sobrecarga de operadores. Já linguagens como Python permite o uso de sobrecarga e até caracteres especiais se necessário.

Conversões de tipos: alargamento, estreitamento e coerção:

Neste contexto a coerção de tipos diz respeito a um tipo de variável que precisa aumentar seu tamanho dinamicamente de acordo com o contexto do que será executado.

O alargamento ocorre quando um valor de tipo menor é convertido por um tipo que possui capacidade maior de capacidade de representação de faixa numérica, por exemplo, int para float.

O estreitamento é o oposto do alongamento, é quando um valor maior precisa ser convertido para um tamanho menor, ou uma faixa de representação mais restrita.

A coerção é a análise realizada automaticamente pelo compilador ou interpretador da linguagem quando diferentes tipos de dados são combinados em uma operação.

Módulo 3: Conversões, expressões booleanas e curto-circuito

Conversões de Tipos: Estuda as regras de coerção, alargamento e estreitamento de dados nas linguagens.

Expressões Booleanas: Foco na lógica de avaliação de condições verdadeiras ou falsas.

Avaliação de Curto-Circuito (*Short-Circuit*): Mecanismo de otimização em que a segunda parte de uma expressão lógica (como em `A && B`) não é avaliada se o resultado do primeiro operando já for suficiente para definir o desfecho.

Módulo 4: Sentenças de Atribuição

Atribuição Simples e Composta: Estuda como os valores são armazenados e atualizados nas variáveis, utilizando operadores tradicionais (`=`) ou abreviados (`+=`, `-=`).

Atribuições Múltiplas: Explora formas de inicializar ou atualizar várias variáveis simultaneamente em uma única linha, dependendo do suporte da linguagem.

Módulo 5: Seleção, Iteração, Goto e Comandos Protegidos

Controle de Fluxo Condicional e de Repetição: Aborda a base de qualquer lógica computacional usando estruturas de seleção (como o bom e velho `if-else` ou `switch`) para desviar caminhos, e estruturas de iteração (laços de repetição como `while` e `for`) para automatizar processos repetitivos.

Exemplo prático de repetição: Um laço `for` (`int i = 0; i < 5; i++`) que percorre uma lista ou executa uma ação controlada até atingir o limite estipulado.

Desvios e Guardas: Analisa o perigo e o uso problemático do comando de desvio incondicional (`goto`), que pula o fluxo para qualquer parte do código gerando o temido "código espaguete" de difícil manutenção. Em contrapartida, introduz os comandos protegidos (*guarded commands*), conceito onde um bloco de código só é executado se a sua "guarda" (condição prévia) for estritamente validada como verdadeira, garantindo mais segurança ao fluxo do programa.

Módulo 6: Exercício: Encontre o Bug

Prática de Depuração (*Debugging*): Uma etapa focada em exercitar a resolução de problemas, desafiando o aluno a identificar e corrigir falhas lógicas, sintáticas ou semânticas escondidas em trechos de código

```
int total = 10;
if (total = 20) { // BUG: Aqui foi usado um '=' (atribuição) em vez de '=='
    (comparação)
    System.out.println("Igual");
}
.
```